K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering & Technology

ENSP465 — New Age Programming Languages Lab

Capstone Project — Phase I

Project Proposal & System Design

PROJECT TITLE

Console-Based Student Management System in Go

Student Name	Divyanshi
Roll Number	2301730198
Semester	VII (B.Tech CSE)
Academic Year	2026–2027
Section	C
Submission Date	24 October 2026
Faculty Name	Dr. Hemraj Kumawat
Course Code & Name	ENSP465 — New Age Programming Languages Lab

1. Problem Statement & Motivation

1.1 Problem Description

Educational institutions, coaching centres and small training academies must keep track of basic student data: who the student is, which course they are enrolled in, and how they performed in assessments. In many small settings this information is still maintained in paper registers or loosely structured spreadsheets. Records are entered by hand, the same student may be recorded twice under different spellings, and every total, percentage and grade has to be calculated manually.

This manual process is slow and error-prone. A single arithmetic slip changes a student's grade, searching for one record means scanning a whole register, and updating or deleting an entry often leaves stale copies behind. The people affected are administrative staff and teachers who maintain the records, and students whose results depend on those records being correct.

The problem is therefore to provide a small, reliable tool that lets a user create, view, search, update and delete student records through a simple interface, calculates results automatically and keeps the data saved between runs.

1.2 Real-World Context

The project belongs to the domain of command-line (CLI) data-management tools. It models the core create–read–update–delete (CRUD) workflow found in virtually every record-keeping application, such as a school information system, but is scoped to run on any machine with no server, database or network connection. This makes it suitable for lightweight and offline use as well as for learning.

1.3 Motivation

- **Practical value:** automating total, percentage and grade calculation removes manual arithmetic errors and saves staff time.
- **Foundational skills:** CRUD with persistence is the basis of most real software; building it cleanly demonstrates structs, slices, maps, error handling, file I/O and packages in Go.
- **Extensibility:** a well-layered console system can later be extended with a REST API or a GUI without rewriting the core logic.

1.4 Existing Solutions and Their Limitations

Existing tool	Strength	Limitation for this problem
Paper registers	No technology needed	No search, error-prone calculations, hard to update, easily lost

Existing tool	Strength	Limitation for this problem
Spreadsheets (Excel / Sheets)	Flexible, familiar	No input validation by default, duplicate IDs possible, formulas can be overwritten, not menu-driven
School ERP software	Feature-rich	Heavy, costly, needs server/database setup and training; excessive for a small institute
SQL database + front end	Scalable and robust	Requires installation and administration; too complex for the basic need

A lightweight, self-contained executable that enforces validation, computes results automatically and stores data in a portable file fills the gap between a spreadsheet and a full ERP.

2. Proposed Solution & Language Justification

2.1 Project Overview

The Student Management System is a console application with a menu-driven interface. A user can add a student, view all students, search by ID or name, update a record and delete a record. Each record contains student ID, name, age, course and marks; the system calculates total marks, percentage and grade, and saves all data to a local JSON file so that it persists between sessions.

2.2 Language Chosen

Go (Golang) is selected as the single implementation language.

2.3 Language Justification

1. **Single static binary and fast start-up.** Go compiles to one self-contained executable with no runtime or interpreter. The tool can be handed to a school office and run immediately by copying one file, which fits the “simple and efficient” objective.
2. **Strong static typing with structs.** A student is naturally a struct with typed fields (int, string, float64). The compiler rejects type misuse at build time, and struct tags (`json:"name"`) map the struct to the file format with no extra code.
3. **Built-in, explicit error handling.** Go functions return an error value which must be checked. This suits an input-driven application where invalid IDs, non-numeric marks, missing files and corrupt data are expected cases, and results in clear, recoverable messages rather than crashes.

4. **Rich standard library for persistence and I/O.** The packages encoding/json, os, bufio and sort cover reading console input, serialising records to disk and ordering output. No third-party dependency is needed.
5. **Concurrency-ready design.** Although Phase II is single-user, Go's goroutines, channels and sync.Mutex allow a later extension (for example, a background auto-save or a multi-client server) without changing the language or architecture.
6. **Simplicity and built-in tooling.** Go's small syntax, gofmt, go vet and the built-in testing package make the code readable and testable, which supports the Phase III testing deliverable.

2.4 Key Libraries

Package (standard library)	Purpose
fmt, bufio, os	Console menus, reading user input, formatted output
encoding/json	Saving and loading student records to students.json
strconv, strings	Converting and cleaning user input
sort	Sorting records by ID or name
errors	Defining custom errors (e.g. ErrDuplicateID, ErrNotFound)
testing	Unit tests for the calculator, validator and service layers

Only the Go standard library is planned; no external frameworks are required.

3. Objectives & Scope

3.1 Objectives

1. Implement the five CRUD operations (add, view, search, update, delete) through a menu-driven console interface, with all five operations working end-to-end by the Phase II demonstration.
2. Calculate total marks, percentage and grade automatically for every student, with 100% agreement with hand-computed results on a test set of at least 10 sample students.
3. Persist records in a JSON file so that 100% of saved records are restored correctly after the program is restarted.
4. Validate all input (unique ID, age range 5–100, marks 0–maximum, non-empty name and course) so that invalid input never corrupts the stored data and always produces a clear error message.

5. Deliver unit tests covering at least 70% of the calculator, validator and service code by Phase III.

3.2 In-Scope Features (Phase II and III)

- Add a student with ID, name, age, course and subject marks.
- View all students in a formatted table, sorted by ID.
- Search a student by ID or by (partial) name.
- Update any field of an existing record; delete a record after confirmation.
- Automatic total, percentage and grade calculation.
- JSON file persistence with automatic load at start-up and save after every change.
- Input validation and user-friendly error messages.
- Unit tests, user guide and README (Phase III).

3.3 Out of Scope

- No graphical user interface and no web front end.
- No login, authentication or role-based access control.
- No relational database or network/cloud storage.
- No attendance, fee, timetable or report-card PDF generation.
- No multi-user concurrent access (single local user only).

4. System Architecture & Design

4.1 High-Level Architecture

The system follows a layered design so that each module has one responsibility: a presentation layer talks to the user, a service layer holds the business rules, and a storage layer handles the file. Validation and result calculation are kept in separate modules so they can be tested in isolation.

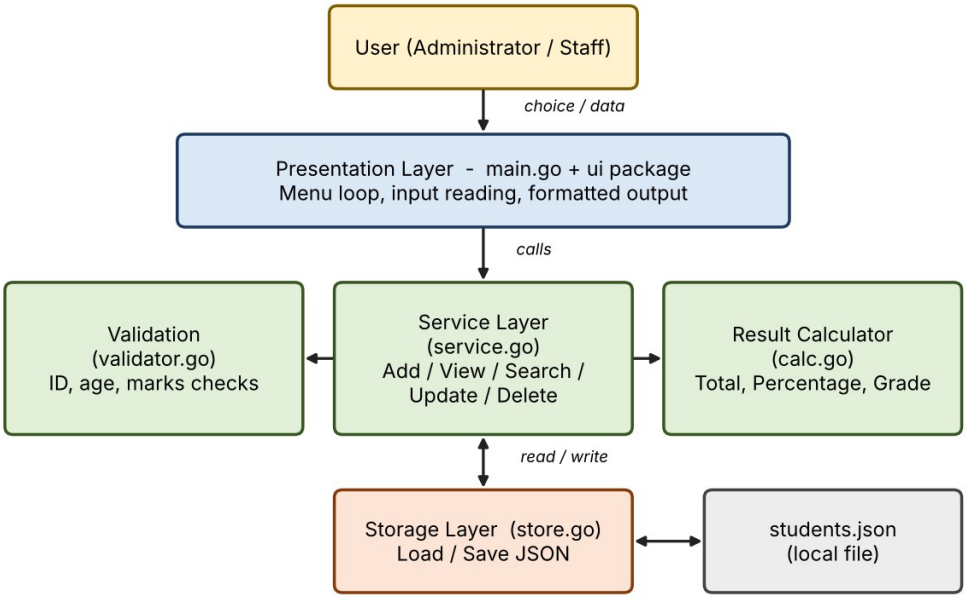


Figure 1: High-level architecture of the Student Management System

4.2 Component Description

- **Presentation layer (main.go, ui):** displays the menu, reads input with bufio.Scanner and prints results as aligned tables. Input: user choices and field values. Output: formatted text and messages. It contains no business logic.
- **Validation module (validator.go):** checks that the ID is a positive and unique integer, the name and course are non-empty, the age lies in range and marks lie between 0 and the maximum. Input: raw field values. Output: nil or a descriptive error.
- **Service layer (service.go):** implements AddStudent, ListStudents, SearchByID, SearchByName, UpdateStudent and DeleteStudent over an in-memory collection. Input: validated student data. Output: updated collection or an error such as ErrNotFound.
- **Result calculator (calc.go):** computes total = sum of marks, percentage = total ÷ maximum × 100, and the grade. Input: marks slice. Output: total, percentage, grade.
- **Storage layer (store.go):** Load() reads students.json into memory at start-up (creating an empty file if absent); Save() writes the full collection after each modification, using a temporary file and rename to avoid partial writes.

4.3 Data Flow Diagram (Level 1)

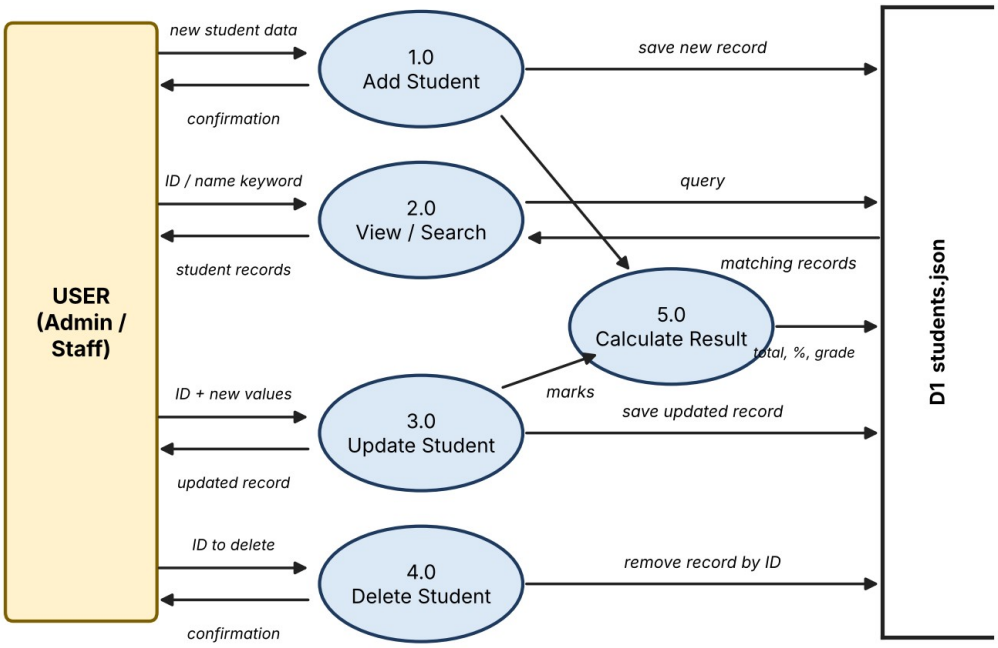


Figure 2: Data Flow Diagram - Level 1

The user supplies menu choices and student data to the five processes. Processes read from and write to the single data store D1 (students.json). Process 5.0 is invoked whenever marks are created or changed, so that total, percentage and grade are always consistent with the stored marks.

4.4 Sequence Diagram — Add Student

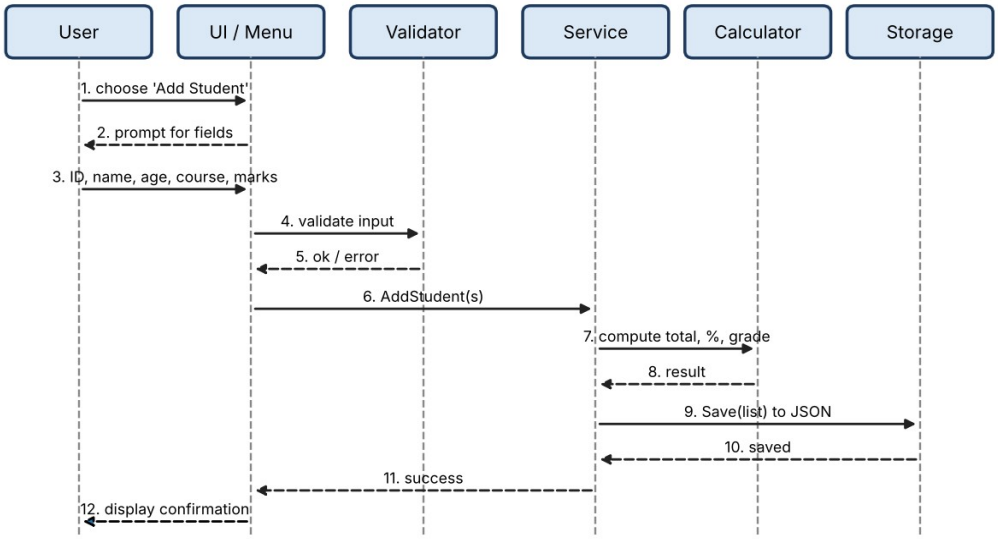


Figure 3: Sequence diagram - Add Student workflow

4.5 Input–Process–Output (IPO)

Stage	Description	Example
Input	Menu choice (1–6); student ID, name, age, course, marks per subject; search keyword	ID 101, “Aman Verma”, 21, “B.Tech CSE”, marks 78, 85, 90, 72, 88
Process	Validate input → apply operation on the in-memory list → compute total, percentage and grade → save list to JSON	Total = 413; Percentage = 82.6%; Grade = A
Output	Confirmation or error message; formatted student table; updated students.json	“Student added successfully.” and a record row

4.6 Grading Scheme

The percentage is mapped to a grade using the following scheme (configurable in calc.go).

Percentage	Grade
90 and above	O (Outstanding)
80 – 89.99	A
70 – 79.99	B
60 – 69.99	C
50 – 59.99	D
Below 50	F (Fail)

4.7 Key Data Structures and Go Features

- **Struct:** type Student struct { ID int; Name string; Age int; Course string; Marks []float64; Total float64; Percentage float64; Grade string } with JSON struct tags.
- **Slice and map:** a slice []Student preserves order for display, while a map[int]int index (ID → position) gives O(1) duplicate detection and lookup.
- **Packages:** the code is organised into packages (ui, service, store, calc, validator) following Go’s package conventions.
- **Custom errors:** sentinel errors and fmt.Errorf with %w wrapping so callers can test with errors.Is.

- **Methods and interfaces:** a Store interface (Load, Save) allows a different backend or an in-memory mock in tests.
- **Defer and file I/O:** defer file.Close() guarantees resources are released on all paths.
- **Table-driven tests:** the testing package with table-driven cases for grade boundaries and validation rules.

5. Work Plan & Timeline

This is an individual project. The plan below maps work from Phase I submission to the final Phase III submission. Dates are tentative and will be aligned with the schedule announced by the faculty.

No.	Milestone	Deliverables	Tentative date
1	Phase I submission	Proposal and system design document (this document)	24 Oct 2026
2	Setup and core model	Go module, project structure, Student struct, calculator and validator with unit tests	1st week of Nov 2026
3	CRUD and persistence	Service layer, JSON storage, add/view/search/update/delete working	2nd week of Nov 2026
4	Phase II: console UI and demo	Complete menu-driven interface, incorporation of Phase I feedback, intermediate demonstration	3rd week of Nov 2026
5	Testing and refinement	Unit and manual test cases, bug fixes, edge-case handling, code clean-up	1st week of Dec 2026
6	Phase III: final deliverable	Final source code, README and user guide, test report, viva preparation	2nd week of Dec 2026

Phase-wise Delivery

- **Phase II (Implementation & Intermediate Demonstration):** milestones 2–4 — working application with all CRUD operations, calculations and JSON persistence.
- **Phase III (Final Deliverable, Testing & Viva):** milestones 5–6 — tested and documented final product and viva.

Task distribution: as an individual project, the student is responsible for all modules (UI, service, storage, calculator, validator and testing).

6. References

References are in IEEE style.

1. The Go Authors, “The Go Programming Language Documentation,” go.dev. [Online]. Available: https://go.dev/doc/. Accessed: October 2026.
2. The Go Authors, “Effective Go,” go.dev. [Online]. Available: https://go.dev/doc/effective_go. Accessed: October 2026.
3. The Go Authors, “Package json — encoding/json,” Go Standard Library. [Online]. Available: https://pkg.go.dev/encoding/json. Accessed: October 2026.
4. The Go Authors, “Package testing,” Go Standard Library. [Online]. Available: https://pkg.go.dev/testing. Accessed: October 2026.
5. A. A. A. Donovan and B. W. Kernighan, The Go Programming Language. Boston, MA, USA: Addison-Wesley, 2015.
6. Course material, ENSP465 — New Age Programming Languages Lab, School of Engineering & Technology, K.R. Mangalam University.